

Exercise 2

SQL Aggregate functions and Operators

Students Table

Students

Student_id	name	age	department
1	Alice	20	IT
2	Bob	22	HR
3	Charlie	21	IT
4	Diana	23	Finance
5	Eve	22	HR

1. List all distinct departments in the Students table

```
SELECT DISTINCT(department)
FROM students;
```

department
IT
HR
Finance

1. The DISTINCT eliminates the repetition.

2. Get the average age of Students per department

```
SELECT department,
    AVG(age) AS AVG_age
FROM students
GROUP BY department;
```

department	AVG-age
IT	21
HR	22
Finance	23

3. Show departments with more than 1 student

```

SELECT department,
       COUNT(*) AS student_count
FROM students
GROUP BY department
HAVING COUNT(*) > 1;

```

department	student_count
IT	2
HR	2

4. Get all students whose age is between 21 and 23.

```

SELECT student_id,
       name,
       age,
       department
FROM students
WHERE age BETWEEN 21 AND 23;

```

student_id	name	age	department
2	Bob	22	HR
3	Charlie	21	IT
4	Diana	23	Finance
5	Eve	22	HR

Q5. List all students in the IT or HR department who are older than 21

```
SELECT student_id,  
       name,  
       age,  
       department
```

```
FROM student
```

```
WHERE department = 'IT' OR 'HR' AND age > 21;
```

Student_id	name	age	department
2	Bob	22	HR
5	Eve	22	HR

∴ It filters out IT department because both students' age do not meet the >21 requirement.

Section 2

Courses Table

Courses

course_id	Course-name	department	Credits
101	SQL Basics	IT	3
102	Python	IT	4
103	Data Science	IT	4
104	Excel	Finance	2
105	Statistics	HR	3

6 Show total Credits per department, Only for departments with more than 5 total credits

```
SELECT department,  
       SUM(credits) AS total_credits  
FROM courses  
GROUP BY department  
HAVING SUM(credits) > 5;
```

department	total_credits
IT	11

Only IT will be chosen as the total credits is > 5 .

7. List all courses that do not have 4 credits.

```
SELECT course_id,  
       course_name,  
       department,  
       credits
```

```
SELECT *  
FROM courses  
WHERE NOT credits = 4;
```

```
FROM courses  
WHERE NOT credits = 4;
```

course_id	course_name	department	credits
101	SQL Basics	IT	3
104	Excel	Finance	2
105	Statistics	HR	3

8. Show the top 3 courses by credits in descending order

```
SELECT course_id,  
       course_name,  
       credits  
FROM courses  
ORDER BY credits DESC  
LIMIT 3;
```

course_id	course_name	credits
103	Data Science	4
102	Python	4
101	SQL Basics	3

Section 3

Enrollments Table

enrollments

enrollment_id	Student_id	course_id	grade
1	1	101	85
2	2	102	78
3	3	103	90
4	4	104	88
5	5	105	82

09. Get the maximum, minimum, and average grade across all enrollments.

```
SELECT Max(grade) AS max_grade,  
       Min(grade)
```

```
SELECT MAX(grade) AS max_grade,  
       MIN(grade) AS min_grade,  
       AVG(grade) AS avg_grade  
FROM enrollments;
```

max_grade	min_grade	avg_grade
90	78	85

10. Count how many enrollments exist per course

```
SELECT course_id,  
       COUNT(*) AS enrollment_count  
FROM enrollments  
GROUP BY course_id;
```

course_id	enrollment_count
101	1
102	1
103	1
104	1
105	1

Section 4.

Salaries Table

Salaries

employee_id	name	department	salary	bonus
1	Tom	IT	60000	5000
2	Jerry	HR	55000	4000
3	Splire	Finance	70000	6000
4	Tyke	IT	62000	5500
5	Butch	HR	54000	3500

11. Find total salary and total bonus per department

```

SELECT department,
       SUM(salary) AS total_salary,
       SUM(bonus) AS total_bonus
FROM salaries
GROUP BY department;
    
```

department	total_salary	total_bonus
IT	122000	10500
HR	109000	7500
Finance	70000	6000

12. Show departments where average salary is above 55000

```

SELECT department,
       AVG(salary) AS avg_salary
FROM salaries
GROUP BY department
HAVING AVGAVG(salary) > 55000
    
```

department	avg_salary
IT	61000
Finance	70000

13. List employees whose salary plus bonus is greater than 60 000

```
SELECT employee_id,  
       name,  
       salary,  
       bonus,  
       salary + bonus AS total-compensation  
FROM salaries  
WHERE salary + bonus > 60 000;
```

employee_id	name	salary	bonus	total-compensation
1	Tom	6000	5000	65000
3	Spike	7000	6000	76000
4	Tyke	62000	5000	67500

Section 5

Projects Table

projects

project_id	project_name	department	budget
1	AI App	IT	120 000
2	Payroll system	Finance	80 000
3	Dashboard	IT	150 000
4	Website	Marketing	60 000
5	HR Portal	HR	50 000

14. SELECT department,
 SUM(budget) AS total-budget
 AVG(budget) AS avg-budget
 FROM projects
 GROUP BY department
 HAVING AVG(budget) > 70 000;

department	total-budget	avg-budget
IT	270 000	135 000
Finance	80 000	80 000

15. SELECT *
 FROM projects
 (WHERE budget BETWEEN 50 000 AND 120 000 AND
 department NOT IN('marketing'));

project-id	projectname	department	budget
1	AI App	IT	120 000
2	Payroll System	Finance	80 000
5	HR Portal	HR	50 000